Please upload the source code of your solutions on or before the due date to the provided Moodle assignment as a single zip file using your group id as the file name. Provide some brief instructions on how to run your solution to each problem in a file called `Problem_X.txt`*, and report also the most important results (such as number of results, runtimes, etc.) of your solutions to that problem within this file. Remember that all solutions may be submitted in groups of up to 3 students.*

New for 2026: Following the University Guidelines for the usage of Generative AI in teaching and learning, we explicitly encourage the usage of AI to solve this exercise sheet. Please indicate which AI platform you used and also summarize the most important prompting steps (if any) together with your solution to each problem. All solutions will be checked for completeness and their functionality!

***Note:* Since this exercise sheet imposes intensive computational workloads, it is highly recommended to run the following experiments on the IRIS HPC cluster by using either the interactive or batching modes of the Spark launcher scripts (see once more the "Getting Started" guide provided on Moodle for detailed instructions).**

Predicting Heart Diseases via Decision Trees & Random Forests **12 Points**

Problem 1. Consider the "*Key Indicators of Heart Disease*" dataset as it is available from Kaggle:

`https://www.kaggle.com/datasets/kamilpytlak/personal-key-indicators-of-heart-disease`

You may download the dataset (in CSV format) either from the above URL or via Moodle. This openly available dataset captures 18 variables (9 booleans, 5 strings and 4 decimals) related to possible indicators of heart diseases from about 320K patients in the U.S. The first (Boolean) variable captures whether a patient suffers from a *HeartDisease* (Yes/No) and should be used as the target label for the following supervised classification steps. Follow the general methodology provided in the `RunDecisionTrees.scala` or `RunDecisionTrees.ipynb` notebooks to solve the following tasks.

(a) Load the entire CSV file into a corresponding DataFrame either directly via `spark.read.option("inferSchema", true).option("header", true).csv(...)` or `spark.createDataFrame(rdd).toDF(...)` from an initial RDD within your Spark session. In case you chose the latter option, also explicitly apply a proper schema (see the header of the CSV file) to this DataFrame with meaningful attribute names for this dataset, either by providing the attribute names directly within the `toDF(...)` method or by defining a new `StructType` with respective `StructField` entries to provide the attribute names and their datatypes[1]. **2 Points**

(b) In the next step, assume we wish to include the `CrossValidator` API of Spark in order to more systematically learn the hyper-parameters of a DecisionTree classifier. To do so, extend the given notebooks by the following steps: **6 Points**

(i) Instead of applying a static 80%/10%/10% split, first create a random split of the DataFrame you constructed in (a) into 80% training and 20% testing data. Define a basic *ML Pipeline*[2] consisting of the necessary stages to train a DecisionTree model by using the 80% split as training data.

(ii) Consider the *ML Tuning*[3] guide of Spark to implement a fully automatic hyper-parameter tuning step into your script. Specifically, use your pipeline to initialize a 5-fold cross-validation loop (using the `CrossValidator` API) over a fixed hyper-parameter setting (using `impurity="entropy","gini"`, `depth=6,12,24` and `bins=20,50,100` as parameter ranges).

(iii) Instead of the `MulticlassMetrics` package used in the provided Moodle script, use the `BinaryClassificationMetrics` API in your pipeline to measure the average accuracy over the two possible labels of *HeartDisease* for each of the cross-validation and hyper-parameter

[1] See: `https://spark.apache.org/docs/latest/sql-programming-guide.html`
[2] See: `https://spark.apache.org/docs/latest/ml-pipeline.html`
[3] See: `https://spark.apache.org/docs/latest/ml-tuning.html`

iterations. Then fit the model to the training data (caution: this step is expensive!), and finally save the best obtained DecisionTree model into a file.

(iv) Finally, measure *precision*, *recall* and *accuracy* over the two possible labels of *HeartDisease* for your best ML model and hyper-parameter setting over the 20% test split you created in (i) by using again the `BinaryClassificationMetrics` API of Spark.

(c) Replace the `CrossValidator` API in your pipeline by the simpler `TrainValidationSplit` API (using `trainRatio = 0.8`), and repeat Steps (i)–(iv) from above to compare the results obtained from the two APIs. **2 Points**

(d) Finally, switch to *AgeCategory* as your target label and repeat the methodology described in (b) by training a RandomForest classifier with the `MulticlassMetrics` API instead of using a single DecisionTree classifier with the `BinaryClassificationMetrics` API. Investigate the additional range of `numTrees=5,10,20` to also find the *best amount of trees* (using `"automatic"` training mode) in your RandomForest classifier. Optionally also compare the performance of your RandomForest classifier with a GradientBoostedTree classifier in Spark. **2 Points**

Summarize your results and the Spark runtimes of this experiment in your `Problem_1.txt` file.

Weather Prediction via Linear Regression & Regression Trees **12 Points**

Problem 2. The *Integrated Surface Data* (ISD), available at: `http://www1.ncdc.noaa.gov/pub/data/noaa`, of the National Oceanic and Atmospheric Administration (NOAA) provides a rich collection of weather measurements from around the world. The Air Force station in Luxembourg is also included with all its recordings between 01/01/1949 to 01/04/2025 (these are all files in the subdirectories that start with the USAF identifier `065900`). For a detailed file format and field explanation, read also the specifications provided in `ish-format-document.pdf`.

(a) Apply the provided parsing function `parseNOAA.scala` to load the input files into an appropriate RDD structure in order to then train different *regression models* that can predict the temperature in Luxembourg for a given date and time of the day. Following the instructions from the specification, you will only need to extract the following fields from the input files

4. `GEOPHYSICAL-POINT-OBSERVATION date` (split into three fields: `year`, `month` and `day`)
5. `GEOPHYSICAL-POINT-OBSERVATION time` (extract only the hour without the minutes)
7. `GEOPHYSICAL-POINT-OBSERVATION latitude coordinate`
8. `GEOPHYSICAL-POINT-OBSERVATION longitude coordinate`
10. `GEOPHYSICAL-POINT-OBSERVATION elevation dimension`
13. `WIND-OBSERVATION direction angle`
16. `WIND-OBSERVATION speed rate`
18. `SKY-CONDITION-OBSERVATION ceiling height dimension`
22. `VISIBILITY-OBSERVATION distance dimension`
26. `AIR-TEMPERATURE-OBSERVATION air temperature`
28. `AIR-TEMPERATURE-OBSERVATION dew point temperature`

thus considering the `air temperature` as your target attribute. Be cautious about parsing floating point values and scaling factors properly. Cache the RDDs to avoid the need of re-processing the input files every time you run a model. **2 Points**

(b) Train a LinearRegression model (similar to the `LinearRegression.scala` or `LinearRegression.ipynb` notebooks provided on Moodle) by splitting the dataset into a (i) *training set* with all records from 1949 until 2023 and a (ii) *validation set* with all measurements for 2024. **3 Points**

(c) Train a RandomForest regressor based again on $n = 10$ regression trees and automatic training mode (similar to the `RunDecisionTrees.scala` or `RunDecisionTrees.ipynb` notebooks provided on Moodle) by again splitting the dataset into a (i) *training set* with all records from 1949 until 2023 and a (ii) *validation set* with all measurements for 2024. **3 Points**

Hint: See the Spark API for a reference on how to use the regression-based variant of decision trees: `https://spark.apache.org/docs/3.5.4/ml-classification-regression.html`

(d) Compare your previously trained regression models from (b) and (c) by predicting the temperatures for the Luxembourg weather station along the whole year of 2025 (so this is your *test set*). Calculate the *mean-squared error* between the predicted temperatures and the actual measurements for 2025 (on those dates available from NOAA for 2025) for each of the two regression models. **2 Points**

(e) Which of the above attributes do you think has the *highest correlation* with our target attribute, i.e., the `air temperature`? **2 Points**

Summarize your results and the Spark runtimes of this experiment in your `Problem_2.txt` file.

Recommender Systems via Matrix Factorization **12 Points**

Problem 3. Consider the AudioScrobbler dataset together with the `RunRecommender.scala` or `RunRecommender.ipynb` notebooks provided on Moodle as basis to solve this exercise. Also here, we will follow up on the idea of extending the provided shell script via hyper-parameter tuning and cross-validations.

(a) Before we can invoke the actual hyper-parameter tuning steps, we need to define a proper quality measure for our resulting recommender engine. To do so, we first need to modify the construction of the training and test sets as follows.

(i) Instead of using the entire dataset to create the `trainData` RDD (including the resolved artist aliases), first create a new RDD, called `trainData100`, by filtering the initial `trainData` RDD, as it is created by the provided notebooks, such that it contains `Rating` objects of only those users who listened to *at least 100 distinct artists* (these should be 72,748 users in the entire AudioScrobbler dataset).

In Scala, this can be verified via:

```
trainData.map(x => (x.user, 1)).reduceByKey(_+_).filter(x => x._2>=100).count()
```

(ii) Next, create an 80%/20% split of your `trainData100` RDD into two new `trainData` and `testData` RDDs. As before, the 80% split will be used for training and hyper-parameter tuning, while the 20% split will be used for the final evaluation of your model under the best hyper-parameters you found.

Important: to evaluate the recommendations properly, make sure that, for each user in `trainData100`, you put about 80% of the artists this user has listened to into `trainData` while the remaining 20% are put into `testData`. That is, both `trainData` and `testData` will contain exactly the same users (but possibly different artists for each of them).

Then, follow the steps provided in the Moodle script to train a first recommender model by using the `trainData` RDD.

(iii) Next, take all users from the `testData` RDD and compute the top-50 most recommended artists for each of them.

For each of the resulting lists of recommendations, create a new RDD `predictionsAndLabels` (similarly to the Spark guide[4] and the suggested implementation in the Moodle script) consisting of pairs $(\hat{y}, y)$, where $\hat{y}$ is the recommendation value of the respective artist for the current user, and y is a binary value indicating whether the current user has indeed listened to that artist ($y = 1$) or not ($y = 0$) based on the provided `actualArtistsForUser` function.

[4] See: `https://spark.apache.org/docs/latest/mllib-evaluation-metrics.html`

This RDD should be used as input to the `BinaryClassificationMetrics` API of Spark which then computes the AUC value for each user automatically. Finally, compute the *average AUC* over all of the previously selected users and compare this value also with a similarly computed AUC value when using the `predictMostPopular` baseline recommendation function provided in the given scripts. **6 Points**

(b) Next, consider a "manual" search for the best hyper-parameters of your Recommender system. To do so, implement a triple `for` loop (as shown in Chapter 3, Slide 30) in the provided shell script to evaluate your system over the following ranges of hyper-parameters:

- `rank <- Array(10, 25, 50);`
- `lambda <- Array(1.0, 0.1, 0.01);`
- `alpha <- Array(1.0, 10.0, 100.0);`

For each selection of hyper-parameters, compute the AUC measure of your model over the 10% split contained in the `testData` RDD you created in (a). That is, for each of the test users, check whether they indeed listened to a newly recommended artist and use this information for the AUC computation under each choice of hyper-parameters.

To avoid overfitting, generalize this procedure by applying also a 5-fold cross-validation over `trainData`. Report the *AUC*, *precision*, *recall* and *accuracy* of your best resulting model and choice of hyper-parameters over the `testData` split after the cross-validations have finished. **4 Points**

(c) Finally, create a new user profile (with a user id that has not been taken before in AudioScrobbler) and add 5-10 new ratings of existing artist ids (of your choice) to the original `trainData` RDD created by the `RunRecommender.scala` or `RunRecommender.ipynb` notebooks from Moodle. Using the best choice of hyper-parameters you found in (a)-(b), train a new recommender model with this additional user, print the top-25 newly recommended artists for this user, and manually judge the quality of the results according to your own music taste. **2 Points**

Once more, summarize your results and the Spark runtimes of this experiment in your `Problem_3.txt` file.